PORTFOLIO PROJECT SUMMARY

MindBridge

AI-Powered Mental Health Support Chatbot

Android App Backend | FastAPI + MongoDB + PyTorch + WebSockets

---

DEVELOPER	Parth Mahendrabhai Lad
PROJECT TYPE	Independent / Team Project
CORE ROLES	Patient, Doctor
MY FOCUS	Backend Architecture & Real-Time Systems

CONFIDENTIALITY & ACCURACY NOTE

Underlying project data/code is confidential. This summary is written from memory to explain the concept, workflow, and objective — details are illustrative, not exact.

Project Overview

MindBridge is an Android-based AI mental health support application designed to give people a safe, always-available first point of contact when they're struggling emotionally or physically — and to connect them to a real doctor the moment the situation needs one. The app supports two user roles: **Patient**, who describes their issue and talks to the assistant, and **Doctor**, who monitors escalated conversations and steps in when needed. My focus on this project was the backend architecture — the system that keeps conversations flowing in real time, understands what the patient is feeling, and knows exactly when to bring in a human.

> ***Confidentiality Note:*** *The underlying codebase and data for this project are confidential. This document is written from memory to explain the project's purpose, workflow, and my contribution — it is a conceptual summary for portfolio purposes, not exact technical documentation.*

Objective of the Project

- Provide an accessible, judgment-free first point of contact for people facing emotional distress or minor health concerns, available anytime through a mobile app.
- Use an AI assistant to understand the patient's situation through natural conversation and gauge their emotional state, not just their words.
- Ensure that when a situation is serious or risky, the system reacts immediately — handing the conversation to a real doctor rather than letting an AI keep responding.
- Give doctors a live, prioritized view of patients who need attention, instead of an undifferentiated queue.
- Build a responsive, real-time communication layer so patients never feel like they're talking into a void while waiting for a human.

How the App Works — Step by Step

1. Patient opens the app

The patient logs in and starts a conversation describing what they're going through — whether it's a physical symptom like a high fever, or emotional distress like anxiety or low mood.

2. AI chatbot responds first

The AI assistant listens, asks clarifying questions, and tries to understand both the content of what's said and the emotional tone behind it, using a sentiment/emotion detection model trained to recognize a wide range of emotional states.

3. Conversation is continuously assessed

As the conversation continues, the system keeps evaluating how serious or urgent the situation seems — combining what the patient says with the emotional signals detected.

4. Escalation if needed

If the situation looks serious — a medical concern beyond general advice, or language suggesting the person may be in crisis — the system stops relying on the AI and immediately routes the conversation to a live doctor.

5. Doctor takes over

A doctor sees the escalated conversation on their dashboard, reviews the context the AI has gathered, and continues the conversation directly with the patient.

6. Doctor gives guidance

The doctor can reassure and advise the patient directly in the app, or, if the situation needs in-person care, tell the patient to visit a hospital or clinic.

7. Session closes safely

Once the conversation is resolved, the session ends. If a patient starts an escalation and then goes silent, an inactivity safeguard eventually closes the session automatically so it doesn't sit open indefinitely.

Note: The details above are illustrative, written from memory to explain the project's purpose and design — exact implementation specifics are confidential and not guaranteed accurate.

Technology Stack

Layer	Technologies
Backend Framework	Python, FastAPI
Database	MongoDB (user profiles, chat history, sessions, escalation status)
AI / ML	PyTorch, HuggingFace Transformers — multi-label emotion/sentiment model
Real-Time Communication	WebSockets (custom ConnectionManager for patient ↔ AI ↔ doctor messaging)
Platform	Android mobile application

Key Components I Worked On

- **Backend architecture** — structured the FastAPI backend to support two distinct user roles (Patient, Doctor) with different views and permissions.
- **Database schema design** — modeled MongoDB collections to persistently store user profiles, full chat histories, active session state, and escalation status.
- **Emotion detection integration** — connected a PyTorch/HuggingFace Transformers model capable of multi-label emotion classification (around 28 emotional categories) to analyze messages in real time.
- **Real-time messaging pipeline** — built a WebSocket-based ConnectionManager to handle simultaneous, bidirectional communication between the mobile app, the AI, and live doctors.
- **Crisis escalation protocol** — designed logic to detect high-risk language (e.g. mentions of self-harm), immediately pause AI responses, and redirect the patient to a doctor's live dashboard.
- **Inactivity watchdog** — implemented an automatic timeout (around 35 minutes) that closes abandoned escalated sessions so doctors' queues don't get clogged with sessions no one is responding to.

Note: The details above are illustrative, written from memory to explain the project's purpose and design — exact implementation specifics are confidential and not guaranteed accurate.

What This Project Gave Me

- Hands-on experience designing a backend that serves two very different user experiences (patient vs. doctor) from one system.
- A practical understanding of how to integrate a trained ML model into a live, production-style backend rather than just running it in a notebook.
- Experience building real-time, bidirectional communication using WebSockets — a step up from typical request/response REST APIs.
- An appreciation for designing safety-critical logic: when AI should stop talking and a human should take over is a genuinely hard design problem, not just an if-statement.
- Practice with NoSQL schema design — structuring MongoDB collections for chat-style, evolving data rather than rigid relational tables.
- A better sense of how sensitive applications (health, mental health) need extra care around escalation, timeouts, and not leaving a user stuck mid-conversation.

Why This Project Matters

Mental health support is often hardest to access exactly when it's needed most — late at night, in a moment of panic, or when someone isn't sure their concern is "serious enough" to bother a doctor. MindBridge was built around the idea that an AI assistant can absorb that first moment of hesitation, listen without judgment, and then get out of the way the instant a real person is needed. The technical challenge wasn't just building a chatbot — it was building a system that knows its own limits.

Prepared as a conceptual project summary for portfolio purposes. Implementation details, architecture specifics, and figures mentioned are illustrative and written from memory — the original project data and codebase remain confidential.